```
import tensorflow
import tensorflow.keras.datasets as datasets
import tensorflow.keras.layers as layers
import tensorflow.keras.models as models
from tensorflow.keras.utils import to_categorical
import numpy as np

# 1. Load & preprocess (same as Q4)
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32")  / 255.0

y_train = to_categorical(y_train, 10)
y_test  = to_categorical(y_test,  10)

input_shape = (32, 32, 3)

# 2. VGGNet Architecture (VGG-style with blocks)
model = models.Sequential()

# Block 1
model.add(layers.Conv2D(64, (3,3), activation='relu', padding='same', input_shape=input_shape))
model.add(layers.Conv2D(64, (3,3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Block 2
model.add(layers.Conv2D(128, (3,3), activation='relu', padding='same'))
model.add(layers.Conv2D(128, (3,3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Block 3
model.add(layers.Conv2D(256, (3,3), activation='relu', padding='same'))
model.add(layers.Conv2D(256, (3,3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Fully Connected
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(10, activation='softmax'))

model.summary()

# 3. Compile & Train
optim = tensorflow.keras.optimizers.Adam(learning_rate=0.001)
model.compile(loss='categorical_crossentropy',
              optimizer=optim,
              metrics=['accuracy'])

model.fit(x_train, y_train,
          batch_size=128,
          epochs=10,
          validation_split=0.2,
          verbose=1)

# 4. Evaluate
_, acc = model.evaluate(x_test, y_test, verbose=1)
print(f"\nTest Accuracy: {100.0 * acc:.2f}%")
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 32, 32, 64) | 1,792 |
| conv2d_1 (Conv2D) | (None, 32, 32, 64) | 36,928 |
| max_pooling2d (MaxPooling2D) | (None, 16, 16, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 16, 16, 128) | 73,856 |
| conv2d_3 (Conv2D) | (None, 16, 16, 128) | 147,584 |
| max_pooling2d_1 (MaxPooling2D) | (None, 8, 8, 128) | 0 |
| conv2d_4 (Conv2D) | (None, 8, 8, 256) | 295,168 |
| conv2d_5 (Conv2D) | (None, 8, 8, 256) | 590,080 |
| max_pooling2d_2 (MaxPooling2D) | (None, 4, 4, 256) | 0 |
| flatten (Flatten) | (None, 4096) | 0 |
| dense (Dense) | (None, 512) | 2,097,664 |
| dropout (Dropout) | (None, 512) | 0 |
| dense_1 (Dense) | (None, 256) | 131,328 |
| dropout_1 (Dropout) | (None, 256) | 0 |
| dense_2 (Dense) | (None, 10) | 2,570 |

```
 Total params: 3,376,970 (12.88 MB)
 Trainable params: 3,376,970 (12.88 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 28s 55ms/step - accuracy: 0.2406 - loss: 2.0034 - val_accuracy: 0.4311 - val_loss: 1.5470
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.4724 - loss: 1.4285 - val_accuracy: 0.5528 - val_loss: 1.2184
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.5906 - loss: 1.1508 - val_accuracy: 0.6355 - val_loss: 1.0349
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.6567 - loss: 0.9814 - val_accuracy: 0.6870 - val_loss: 0.8933
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.7045 - loss: 0.8559 - val_accuracy: 0.7099 - val_loss: 0.8305
Epoch 6/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.7415 - loss: 0.7546 - val_accuracy: 0.7295 - val_loss: 0.7896
Epoch 7/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.7726 - loss: 0.6597 - val_accuracy: 0.7476 - val_loss: 0.7388
Epoch 8/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.8002 - loss: 0.5837 - val_accuracy: 0.7579 - val_loss: 0.7222
Epoch 9/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.8252 - loss: 0.5109 - val_accuracy: 0.7555 - val_loss: 0.7435
Epoch 10/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.8482 - loss: 0.4457 - val_accuracy: 0.7625 - val_loss: 0.7488
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.7534 - loss: 0.7757
```

Start coding or generate with AI.